University for the Creative Arts

BERLIN SCHOOL OF BUSINESS & INNOVATION

Essay / Assignment Title: Credit Card Fraud Detection using Machine Learning

Programme title: Machine Learning Fundamentals

Name: Sanket Sadashiv Gadge

Year: 2025

CONTENTS

CONTENTS 2
1. INTRODUCTION 5
2. CHAPTER ONE: Traditional Approach 6
3. CHAPTER TWO: Data Collection and EDA 7
Distribution Plots: 9
Box Plots for outliers: 10
Z-Score for Outlier Detection: 11
Feature correlations with Target column: 12
4. CHAPTER THREE: Data Preprocessing 13
Stratified Split during train-test-split: 13
SMOTE 13
Scaling 13
5. CHAPTER FOUR: Model Selection and Design 14
Baseline 14
Tree and Ensemble based Model: 15
6. CHAPTER FIVE: Hyper Parameter Tuning 16
HalvingGridSearchCV 16
Observations 17
1. Logistic Regression 17
2. Random Forest 17
3. XGBoost 18
4. MLP 18
Model Ensembling: 19
Stacking Classifier 20
7. CHAPTER SIX: Anomaly Detection 21
IsolationForest: 21
XGBoost + IsolationForest: 21
8. CHAPTER SEVEN: Model Evaluation 22
Accuracy 22
Precision 22
Recall 22
F1-Score 22
9. CHAPTER EIGHT: Challenges and Limitations 24
Data privacy regulations (GDPR compliance) 24
Real-time processing constraints 24
CONCLUDING REMARKS 25
BIBLIOGRAPHY 26
APPENDIX (if necessary) 27

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the program).

Name and Surname (Capital letters):

SANKET GADGE

Date: 17/06/2025

1. INTRODUCTION

The process of identification and prevention of fraudulent credit card transactions is known as credit card fraud detection. Sometimes, a credit card may get stolen and used to make a fraudulent purchase or withdraw cash using it. This affects both credit card users and the financial company who issued this card. With the rise of AI and advancement in predictive modelling, one can build a fraud detection system which alerts both parties whenever such a transaction occurs in the system in some cases even preventing the transaction to be completed. As mentioned in the report by the European Union, the total value of fraud transactions using cards within the EU amounted to €1.87 billion (European Central Bank, 2021).

Here we will discuss some techniques which can be used to build a machine learning model to detect and predict the possibility of fraudulent transactions. The dataset is taken from kaggle, it contains around ~285k records with 31 columns. The dataset is highly imbalanced, with only 0.173% fraud transactions. Given such a high imbalance, we will try to use models which are not mainly affected by imbalance in classes, we will also use techniques like SMOTE or Stratified sampling to keep the imbalance in check while training. Also, for the evaluation metrics we can't rely on accuracy score, instead we will have to use precision, recall and f1_score to properly evaluate the models. We will also try ensemble techniques to improve the accuracy and deal with the shortcomings of the individual models.

2. CHAPTER ONE: TRADITIONAL APPROACH

Previously, credit card fraud detection relied highly on rule-based systems and manual verification processes before the era of ML and AI algorithms. Financial institutions had to study a lot of patterns up to specific thresholds for payments / transactions made by users and then flag the suspicious ones out of them which needed a whole lot of teamwork from analysts. For example, a card used in London suddenly records high-value transactions in Moscow, the system would automatically freeze the account. Manual reviews by fraud analysts overrides these rulesets by cross-investigating the transaction against historical patterns and already known fraud databases.

Companies like Visa and American Express had already used these traditional approaches at some scale. Visa's fraud systems at the very beginning employed these rule-based systems that monitored the transaction velocity with its merchant category codes and American Express used manual signature verification on receipts generated for in-person transactions.

3. CHAPTER TWO: DATA COLLECTION AND EDA

We are using publicly available dataset which is available kaggle. It has a total of 284,807 total records out of which only 492 records are fraud. It contains following columns:

1. V1-V28 : These are the principal components derived from the actual features. Due to confidentiality issues, the real features were not provided.
2. Amount: Amount is the transaction amount.
3. Time: Time is the elapsed time in seconds from the first transaction. All the given transactions happened in two days of time.
4. Class: our target variable which is either 0 or 1 meaning not fraud or fraud.

After collecting the dataset, we ran a few basic checks on it mainly to find out if there are any missing values (na values), duplicate records in the dataset. There were no missing values, although we found 1081 duplicate records which we removed after careful considerations. After checking the class label distributions again, the ratio between fraud and not fraud record was 0.167% now, which was previously 0.172%. Next, we tried to plot distribution of Time and Amount columns, those were the only two columns on which PCA transformation was not applied.

Figure 1 : KDE plot for Amount and Time

By looking at the plot for the Amount, we can see that its right skewed, which implies that most of the transactions cluster around 0, and there are few transactions with larger amounts. Along

with that we have a time distribution which shows that there are two peaks for two days of records.

One interesting thing we found is that there were 27 records with the transaction amount being 0. As for the reason, it wasn't clear why as the information was not provided by the authors.

It is noticed that the fraudulent transactions that mostly occur for the smaller amounts. We still see the max amount being 2125.87. 99% of the fraudulent transactions were around 1357.13 while 50% being lower than 9.25.

Distribution Plots:

Figure 2 : Distribution plots for all columns from V1 to V28

Most of the features show normal distribution, except a few like V1, V17 which do show some skewness.

Box Plots for outliers:

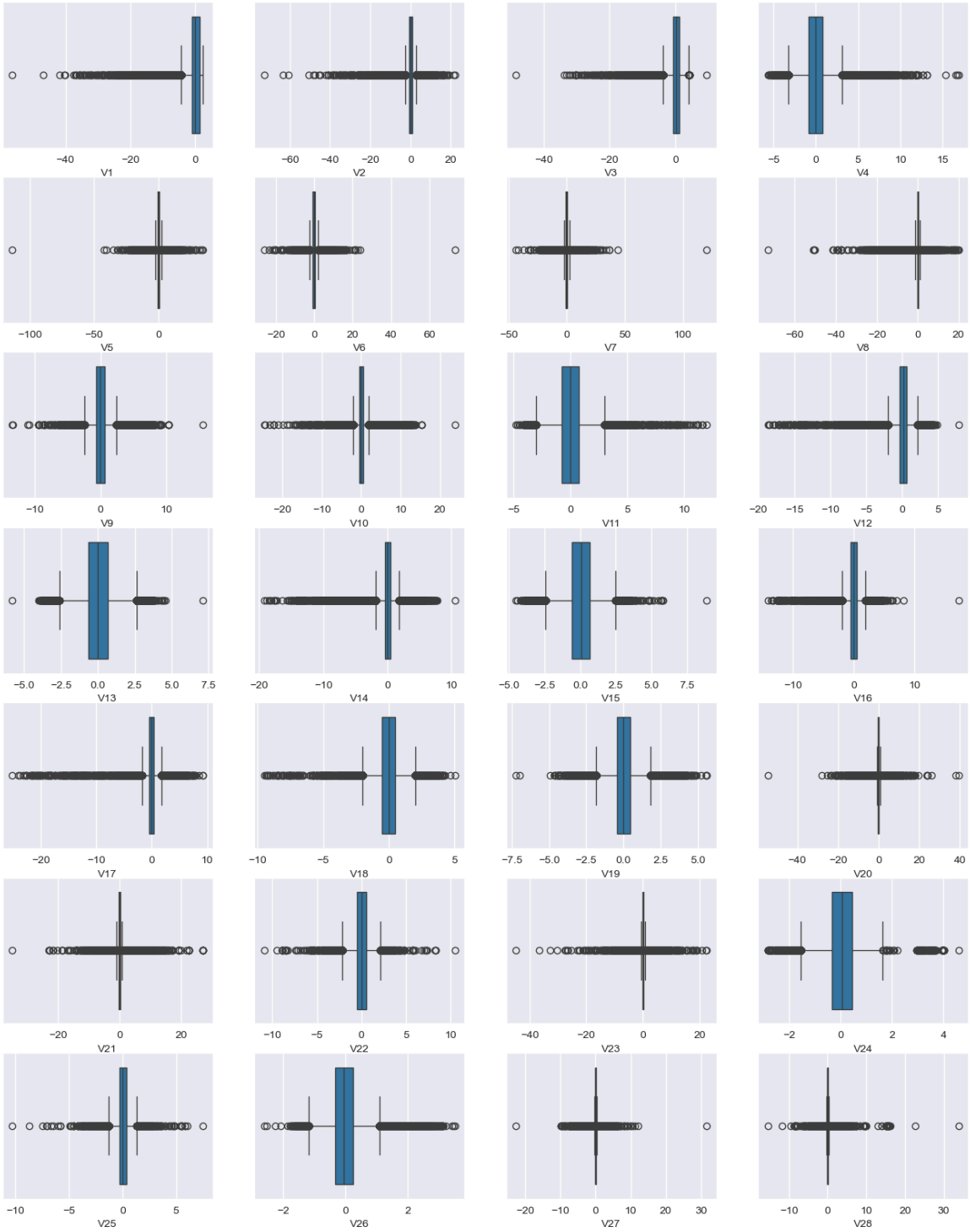

Figure 3 : Box plot for showing outliers for columns from V1 to V28

We tried to plot boxplots, We can see that box plots using IQR method cannot be used on this dataset for the outliers. We will need to find the outliers using a different technique.

Z-Score for Outlier Detection:

As the boxplot method was not useful, we went with the z-score test. With the threshold of 3σ we applied a z-score test on the whole dataset.

```
z_threshold = 3
zscore_df = df.drop(columns=['Class']).apply(zscore)
zscore_df = zscore_df.abs() > z_threshold
zscore_df = zscore_df.any(axis=1)
outlier_rows = df[zscore_df]
outlier_rows['Class'].value_counts()
```

[41]

```
Class
0    37372
1      444
Name: count, dtype: int64
```

Figure 4 : Calculating z-score for all the columns except class

We can see that, only by applying the z-score, we could filter our 444 records out of 473, which is almost 93% of the fraud transactions.

Feature correlations with Target column:

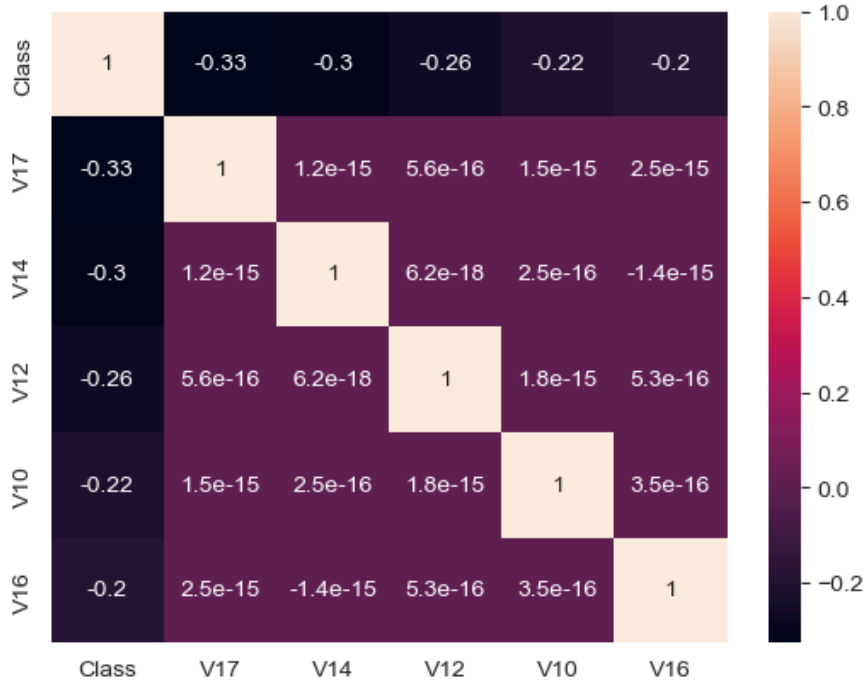

Figure 5 : Heatmap showing feature correlations with respect to column Class

Using the correlation, we can find which of the features have the strongest correlation with our target column meaning which columns will help significantly in our training to classify the given data points.

Above is the correlation matrix of the top 5 correlated features. V17 being the most correlated feature overall. To further strengthen the evidence of the correlation we also ran the Man-Whittney U test along with the Bonferroni adjustment. By running this test we were also able to conclude that features V26, V23, V25, V13, V15 and V22, don't seem to be much impactful on the target column.

4. CHAPTER THREE: DATA PREPROCESSING

Stratified Split during train-test-split:

As we know our dataset is highly imbalanced, thus we are required to pick stratified samples when creating train test splits to maintain the same distribution of data in both train and test set. Here, we also created a separate validation set, this will be used later in order to tune hyperparameters.

SMOTE

To tackle the issue of imbalanced class, we are using SMOTE which is an upsampling technique from the scikit-learn package which basically adds fake samples to our minority class in order to maintain the balance between classes while training. There are few other options like KmeansSMOTE, although the imbalance ratio on this dataset is too huge.

Scaling

Except Amount, Time and Class all other features are gone through PCA transformation. So the only columns which required Scaling were Amount and Time.

5. CHAPTER FOUR: MODEL SELECTION AND DESIGN

Baseline

For the baseline, we went with the classic LogisticRegression Model, with and without SMOTE (left). We also added class_weight = ‘balanced’.

```
Accuracy: 99.90519995786666 %
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     28432
           1       0.81      0.59      0.68        49

    accuracy                           1.00     28481
   macro avg       0.90      0.80      0.84     28481
weighted avg       1.00      1.00      1.00     28481

Accuracy with class_weight=balanced: 97.68266563674028 %
              precision    recall  f1-score   support

           0       1.00      0.98      0.99     28432
           1       0.07      0.94      0.12        49

    accuracy                           0.98     28481
   macro avg       0.53      0.96      0.56     28481
weighted avg       1.00      0.98      0.99     28481
```

Figure 6 : Evaluation metrics for baseline Logistic Regression model without SMOTE

```
Accuracy: 99.16084407148625 %
              precision    recall  f1-score   support

           0       1.00      0.99      1.00     28432
           1       0.16      0.92      0.27        49

    accuracy                           0.99     28481
   macro avg       0.58      0.96      0.63     28481
weighted avg       1.00      0.99      0.99     28481

Accuracy with class_weight=balanced: 99.16084407148625 %
              precision    recall  f1-score   support

           0       1.00      0.99      1.00     28432
           1       0.16      0.92      0.27        49

    accuracy                           0.99     28481
   macro avg       0.58      0.96      0.63     28481
weighted avg       1.00      0.99      0.99     28481
```

Figure 7 : Evaluation metrics for baseline Logistic Regression model with SMOTE

On the very basic model without any hyper parameter tuning we got 99% accuracy. Although if you look closely at the classification report, the recall for the class=1, is very low. Overall the model with balanced class_weights gave us the best recall of 94%, meaning it was able to detect most of the fraud. But its precision score was very low, meaning there were a lot of False positives.

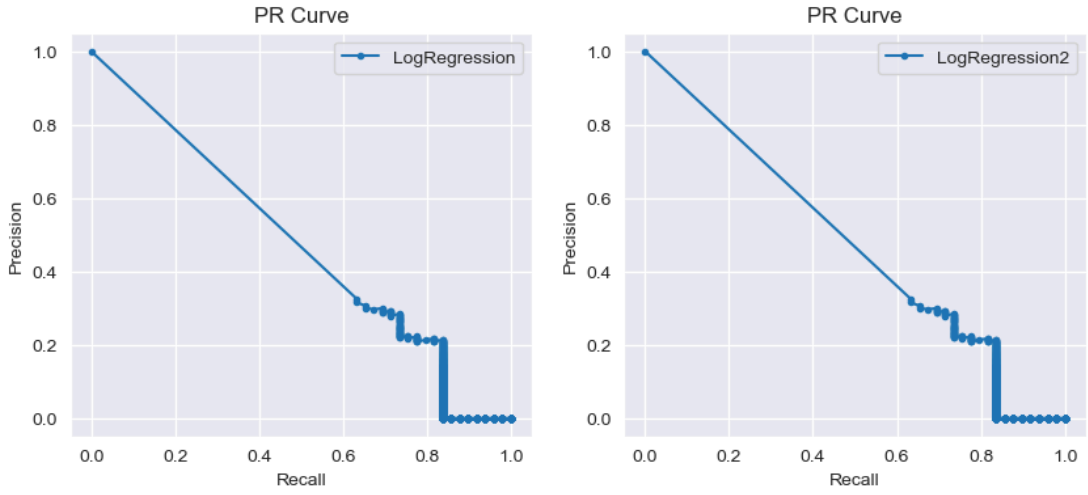

Figure 8 : Precision Recall curve for same above Logistic Regression model.

Tree and Ensemble based Model:

After running the baseline model, we experimented with RandomForest and XGBoost model which comes from tree based architecture families and comes with robustness towards outlier and class imbalance. With both of these models we got a very good f1 score which implies that both precision and recall are good.

```
XgBoost Accuracy: 99.96 %
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     28432
           1       0.90      0.88      0.89        49

    accuracy                           1.00     28481
   macro avg       0.95      0.94      0.94     28481
weighted avg       1.00      1.00      1.00     28481
```

Figure 9 : Evaluation metrics for baseline XGBoost model.

```
RandomForest Accuracy: 99.96 %
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     28432
           1       0.89      0.86      0.88        49

    accuracy                           1.00     28481
   macro avg       0.95      0.93      0.94     28481
weighted avg       1.00      1.00      1.00     28481
```

Figure 10 : Evaluation metrics for baseline Random Forest model.

Here, XGBoost performs slightly better than RandomForest.

6. CHAPTER FIVE: HYPER PARAMETER TUNING

Hyperpameter tuning is a crucial step in any machine learning model training pipeline. It is the process of selecting the best optimal set of parameters that are given to your machine learning model in order to control its learning process(e.g. learning rate, number of layers). This directly affects your model's accuracy and performance so it is very important to choose the right set of hyperparameters. For this step, we choose 4 different models which are LogisticRegression, XGBoost, RandomForest and MLPClassifier.

HalvingGridSearchCV

As opposed to the traditional GridSearchCV, HalvingGridSearchCV on the other hand starts with a small subset of resources and progressively trains and evaluates most promising candidates with increasing resources while eliminating less performing ones. As we noticed that GridSearch was taking a lot of time, we had to use an advanced technique such as this.

Below are the parameters we got from the search:

```
[{'Model': 'LogisticRegression',
  'Best Params': {'clf__class_weight': 'balanced', 'clf__max_iter': 500},
  'Best Score': 0.9061343719571567},
 {'Model': 'RandomForest',
  'Best Params': {'clf__class_weight': None,
   'clf__criterion': 'gini',
   'clf__n_estimators': 200},
  'Best Score': 0.7817267121064589},
 {'Model': 'XGBoost',
  'Best Params': {'clf__learning_rate': 0.01, 'clf__n_estimators': 500},
  'Best Score': 0.7716001298279779},
 {'Model': 'MLP',
  'Best Params': {'clf__activation': 'tanh',
   'clf__hidden_layer_sizes': (10, 20, 50, 100),
   'clf__learning_rate': 'adaptive',
   'clf__max_iter': 200,
   'clf__solver': 'adam'},
  'Best Score': 0.7489126906848427}]
```

Figure 11 : List of best parameters given by Halving GridSearch Cross Validation method of hyperparameter tuning.

By using these parameters, we initialised new model instances and trained them again.

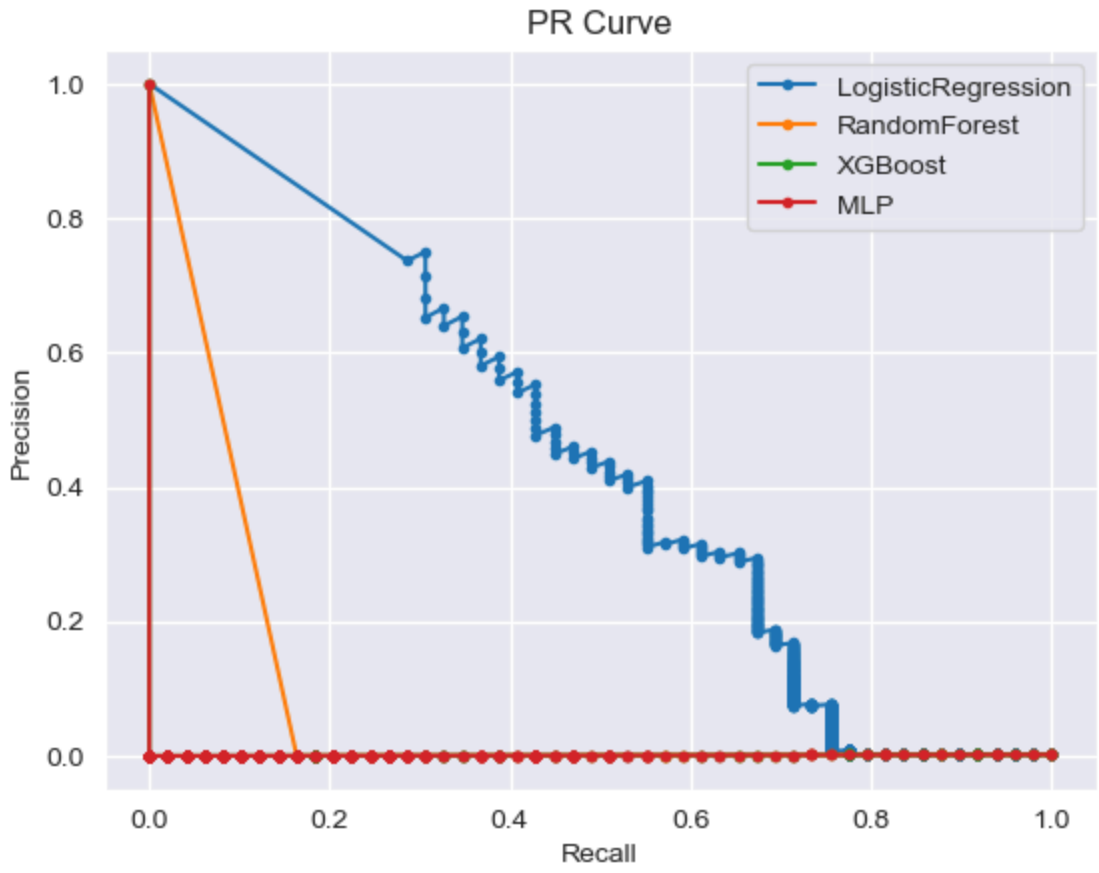

Figure 12 : Precision Recall curve for all the models trained till now.

Observations

1. Logistic Regression

- For Logistic Regression we got 97% accuracy.

- Although the precision and recall for class 1 (Fraud) was 7% and 88% respectively.

- This implies that LR was able to catch 88% of the actual frauds. although precision of 7% implies that there were too many false positives.

2. Random Forest

- For Random Forest we got 99.95% accuracy.

- Precision and recall for class 1, for RF was 95% and 73% respectively.

- The F1 Score came to around 83% which is good. Although, recall is less than logistic regression, meaning it was able to catch less fraud transactions while lowering the false positive count.

3. XGBoost

- For XGBoost we also got 99.95% accuracy.

- Precision and recall for class 1, for RF was 100% and 73% respectively.

- Similar to RandomForest, it was not able to beat LR in catching frauds, although it ended up with no false positives at all.

- We can use a combined model (ensemble) of LR and XGBoost to catch more frauds while limiting the false positives.

4. MLP

- For MLP we got 99.91% accuracy, slightly higher than others.

- Precision and recall for class 1, for MLP was 79% and 69% respectively.

- The F1 score is balanced. Could do better.

Model Ensembling:

We have created an Ensemble of XGBoost and Logistic Regression models. As we saw that Logistic Regression by itself had very good recall but precision was not good on the contrary XGBoost showed very good balance between precision and recall. Thus we decided to ensemble those two, with a voting classifier.

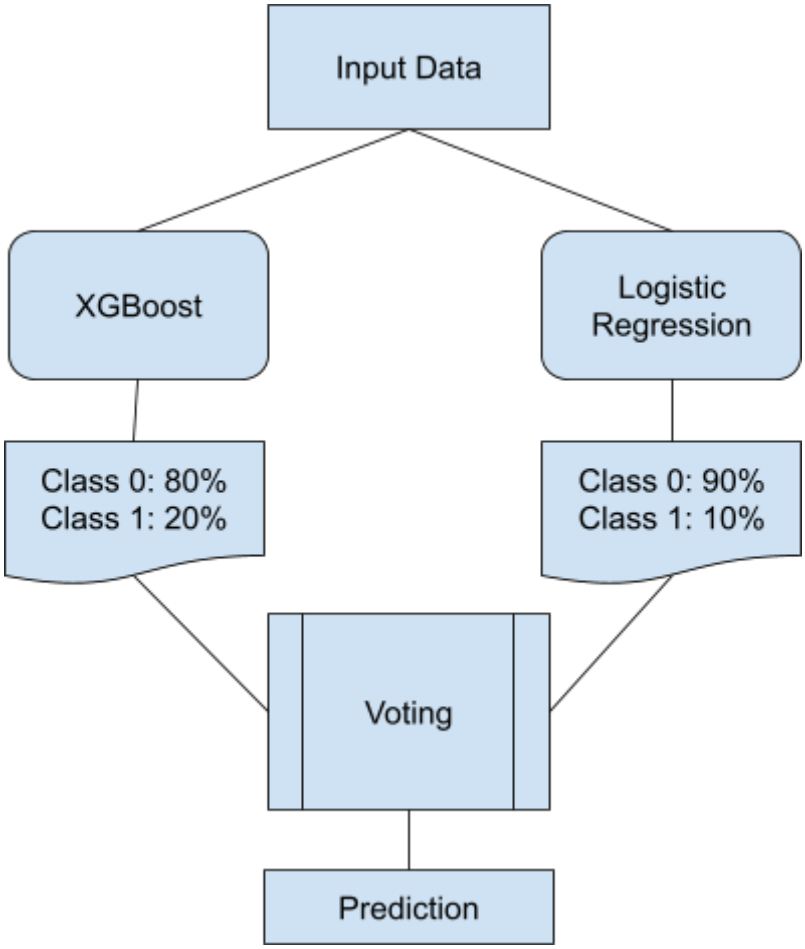

Figure 13 : Soft Voting between XGBoost and Logistic Regression.

Although, when testing this ensemble on the testing set, we were disappointed with the result as the classifier only got 72% f1 score, while we were expecting it to be more.

```
VotingClassifier Accuracy: 0.9989817773252344 %
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     28432
           1       0.68      0.78      0.72        49

    accuracy                           1.00     28481
   macro avg       0.84      0.89      0.86     28481
weighted avg       1.00      1.00      1.00     28481
```

Figure 14 : Evaluation metrics of Voting Classifier

We also trained one more ensemble, this time with XGBoost + RandomForest Classifier. The results were:

```
Final Ensemble Evaluation:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     28432
           1       1.00      0.45      0.62        49

    accuracy                           1.00     28481
   macro avg       1.00      0.72      0.81     28481
weighted avg       1.00      1.00      1.00     28481

Accuracy: 0.9990519995786665
```

Figure 15: Evaluation metrics for Voting Classifier between XGBoost and Random Forest classifiers.

Here, precision score was 100 although recall was low only 45%. In tasks, where having a False Negative could cause financial damages, having high recall is very important.

Stacking Classifier

As the name suggests, we can stack multiple classifiers on top of each other in order to create a true ensemble of multiple models. Here we have stacked XGBoost, RandomForest and MLP classifiers.

```
StackedClassifier Accuracy: 0.9995084442259752 %
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     28432
           1       1.00      0.71      0.83        49

    accuracy                           1.00     28481
   macro avg       1.00      0.86      0.92     28481
weighted avg       1.00      1.00      1.00     28481
```

Figure 16 : Evaluation metrics for Stacked Classifiers of XGBoost, Random Forest and MLP models.

The stacked classifier was able to achieve 100% precision with 71% recall, which is very good.

7. CHAPTER SIX: ANOMALY DETECTION

Before performing anomaly detection , let's know what it means - It is an important step to find unusual patterns also known as outliers in our dataset. These outliers are the records that conform to an expected behaviour which can also be a fraud transaction in the real world as it is also an unexpected transaction. This task of classifying fraud vs non-fraud transactions can also be looked at as an anomaly detection problem, where we try to find clusters of similar data points vs dissimilar ones.

IsolationForest:

The IsolationForest 'isolates' observations by randomly selecting a feature and then randomly selecting a split value between the maximum and minimum values of the selected feature. It returns an anomaly score for each sample, which can be used as a label

XGBoost + IsolationForest:

By adding the anomaly score from IsolationForest in the we could add one more feature which helps our models in detecting the fraud.

```
XGBoost + ISO Forest Accuracy: 0.9989817773252344 %
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     28432
           1       0.68      0.78      0.72        49

    accuracy                           1.00     28481
   macro avg       0.84      0.89      0.86     28481
weighted avg       1.00      1.00      1.00     28481
```

Figure 17 : Evaluation metrics for XGBoost with Isolation Forest models.

Here, we got a 68% precision with 78% recall on our testing set.

8. CHAPTER SEVEN: MODEL EVALUATION

Model evaluation is one of the most important steps in testing a machine learning model. By this we understand how well our machine learning model has performed in different metrics like accuracy, precision, recall , f1-score, etc as shown in below charts. We trained multiple models with multiple techniques like SMOTE and IsolationForest. Here we can see that, The best models in terms of recall are RandomForest, XGBoost which were trained with SMOTE applied on training samples.

Accuracy

To measure overall correctness of the model, we calculate accuracy by dividing the number of correct predictions to the total number of predictions done by the model. Due to dataset imbalance , this metric is not useful here because there is very less fraud transaction data.

Precision

Whatever model has predicted, how positive it is and how correct it is , this is what precision metric defines. We cannot afford false positives because the fraud transaction might be of big amounts. The predicted positives should be actually correct with respect to the total number of transactions.

Recall

Here the model is being compared with the actual data of transactions and how much of them are correctly identified. We cannot afford such cases because what if that was an actual fraud but not correctly identified by the model.

F1-Score

It is a very useful metric for such fraud detection and spam detection cases. We calculate f1-score by double times multiplying precision with recall and then dividing it by addition of precision with recall. So even if one of the metrics is low , the f1 score drops badly which prevents misleading data in the dataset even if the other metric is high. Since both precision and recall are ratios, the calculation is mathematically appropriate.

In this case we have considered f1-score as the best metric to understand our model evaluation. And we can observe that the XGBoost model with SMOTE dataset has the best f1-score , this proves the fact because Boosting algorithms are supposed to give best results regardless of the number of outliers or imbalanced class distribution (Machine Learning Group, n.d. a).

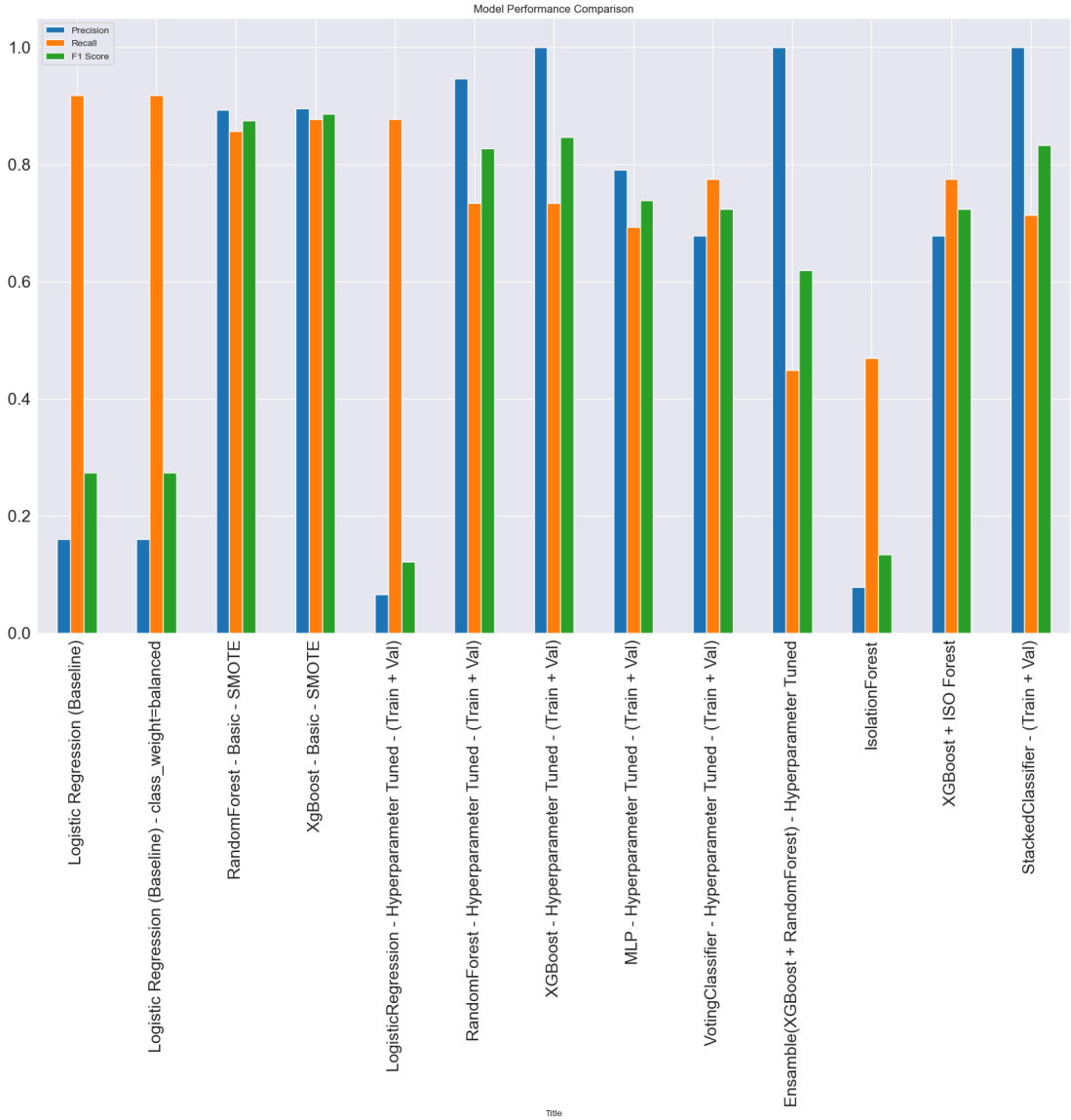

Figure 18 : Bar chart for model performance comparison trained till now.

9. CHAPTER EIGHT: CHALLENGES AND LIMITATIONS

Data privacy regulations (GDPR compliance)

GDPR warns that only mandatory data which is strictly necessary for fraud detection be retained e.g. columns like timestamp, class and amount. But only these 3 columns with PCA applied data makes building the models a lot challenging. Using advanced techniques like differential privacy, synthetic data, federated learning or privacy aware PCA , these can help in keeping a better balance between privacy compliance and model performance.

Real-time processing constraints

The amount of fraud transactions in the real world is very less, near to 0.2% of all transactions. This uniqueness makes it very difficult for companies to even test or distinguish the transactions as fraud. Additionally, people who commit these frauds come up with new tactics or tricks to bypass the security measures often requiring the models to constantly learn a pattern , adapt and evolve.

Also , for such types of fraud detection and prediction systems, due to GDPR and privacy concerns, we have very limited access to real-world fraud data, reducing model training breadth and collaboration in actual scenarios.

CONCLUDING REMARKS

This report has demonstrated an end-to-end machine learning pipeline for credit card fraud detection and prediction, beginning from a traditional rule-based approach to using hyperparameter tuning for the best f1-score of the models. In between, we went through a journey of statistical methods, studying the dataset properly, using techniques like stratified sampling, SMOTE and scaling , then selecting a baseline model and learning through it and again using some tree and ensemble models to again gain some insights of what should be done on the dataset. After conducting a thorough research on the dataset, we finally considered four models named Logistic Regression, Random Forest, XGBoost and MLP for hyperparameter tuning using HalvingGridSearch cross validation technique resulting into a good f1-score for XGBoost with up to 80%. We also studied some issues regarding data privacy and GDPR due to european dataset and some problems that a model might face in going through a real-time set of transactions. For a futuristic approach on such fraud detection datasets, we can make ties with IOT based equipment or mobile devices where a particular service can trigger in background to check the transaction while it is processing. Additionally we can generate such different models for different domains like insurance claims, anti-money laundering in banks and healthcare anomaly detection for early disease recognition. As these are high-stakes domains, the number of fraud data that can get generated increases giving our model flexibility to make good decisions.

BIBLIOGRAPHY

1. European Central Bank (2021). Seventh report on card fraud. European Central Bank, [online] (2021). Available at: https://www.ecb.europa.eu/press/cardfraud/html/ecb.cardfraudreport202110~cac4c418e8.en.html.
2. Machine Learning Group, Université Libre de Bruxelles, n.d. Reproducible Machine Learning for Credit Card Fraud Detection – Practical Handbook. 'Baseline feature transformation' [online]. Available at: https://fraud-detection-handbook.github.io/fraud-detection-handbook/Chapter_3_GettingStarted/BaselineFeatureTransformation.html [Accessed 17 June 2025].

APPENDIX (if necessary)

Github Link:
https://github.com/Sanket758/AtBSBI/blob/master/Machine%20Learning%20Fundamentals%20-%20Assignment/credit_card_fraud_detection.ipynb

Colab Link:

credit_card_fraud_detection.ipynb